

TP1: getcnt syscall on xv6

Instructions on getting Xv6-RISCV running

- Installing dependencies: <https://pdos.csail.mit.edu/6.1810/2023/tools.html>
- xv6 RISC-V repository: <https://github.com/mit-pdos/xv6-riscv.git>

Resources

- Recommended reading: Xv6 book Chapters 1 and 2 plus Sections 4.3 and 4.4.
- Using GDB do help with debugging:
 - See the first part of this page:
<https://pdos.csail.mit.edu/6.1810/2023/labs/syscall.html>
- Similar assignment:
 - <https://github.com/remzi-arpacidusseau/ostep-projects/blob/master/initial-xv6/README.md>

Tasks

1. Implement a getcnt syscall
 - Modify xv6 so the kernel keeps track of how many times each syscall has been called
 - The getcnt syscall receives one integer parameter. It returns the number of times the syscall with the given number has been called. The userspace declaration of getcnt should be `[int getcnt(int)]`
 - Relevant files
 - syscall.c
 - syscall.h
 - sysfile.c
 - proc.h (for trapframe)
2. Implement a user-space program to call the syscall. The program should be named "getcnt" and receive a single integer as parameter corresponding to the target syscall.
 - Relevant files
 - user/user.h
 - user/usys.pl (auto-generates user/usys.S)
 - Makefile (add your program to the UPROGS variable)
 - Example execution (22 is the number of the getcnt syscall in this implementation)

```
$ ./getcnt 1
```

```
syscall 1 has been called 4 times
$ ./getcnt 22
syscall 22 has been called 2 times
$ ./getcnt 22
syscall 22 has been called 3 times
$ ./getcnt 22
syscall 22 has been called 4 times
```

What to submit

You should upload a zip containing the complete version of your modified xv6 with the getcnt syscall. You should also upload a documentation in PDF format with *at least* the following information:

- Show and discuss four (4) modifications you made to the xv6 code; use the [git diff] command to compute the changes between the original xv6 code and your modified version.
 1. Show the diff and discuss the data structure you use to keep track of the number of calls to each syscall
 2. Show the diff and discuss how you updated the data structure
 3. Choose two more parts of the code you find relevant to discuss using diffs
- How you tested your syscall